

✓ Stage 4 — Data Validation & Cleaning

Objective

The objective of this stage is to validate the extracted transaction data and identify potential data-quality issues before performing aggregation and analysis.

The validation process checks data types, missing values, duplicate records, invalid numerical values, fraud-flag values, and categorical values.

Where a correctable data-quality issue is identified, the required cleaning operation is applied while preserving the structure of the transaction data.

Operations Performed

1. Loaded the transaction dataset.
2. Inspected important data types.
3. Converted the timestamp field into datetime format.
4. Checked for missing values.
5. Checked and removed duplicate records when present.
6. Checked for negative transaction amounts.
7. Validated the fraud flag values.
8. Inspected transaction types and transaction statuses.
9. Performed final data-quality validation.

Output

A validated and cleaned dataframe is produced for the next stage.

The resulting dataset is used for aggregation and representation without carrying forward identified data-quality problems.

```
# =====
# STAGE 4 – CONTAINER 1
# INITIAL DATASET VALIDATION
# =====

import pandas as pd
import numpy as np

df = pd.read_csv("upi_transactions_2024.csv")

print("INITIAL DATASET")
print("=" * 60)

print(f"Rows      : {df.shape[0]:,}")
print(f"Columns   : {df.shape[1]:}")

INITIAL DATASET
=====
Rows      : 250,000
Columns   : 17
```

```

# =====
# STAGE 4 – CONTAINER 2
# DATA-TYPE VALIDATION
# =====

print("DATA-TYPE VALIDATION")
print("=" * 60)

print("Before cleaning:")
print(f"timestamp      : {df['timestamp'].dtype}")
print(f"amount (INR)    : {df['amount (INR)'].dtype}")
print(f"fraud_flag       : {df['fraud_flag'].dtype}")

# Convert timestamp to datetime
df["timestamp"] = pd.to_datetime(
    df["timestamp"],
    errors="coerce"
)

# Convert amount to numeric
df["amount (INR)"] = pd.to_numeric(
    df["amount (INR)"],
    errors="coerce"
)

# Convert fraud flag to numeric
df["fraud_flag"] = pd.to_numeric(
    df["fraud_flag"],
    errors="coerce"
)

print("\nAfter cleaning:")
print(f"timestamp      : {df['timestamp'].dtype}")
print(f"amount (INR)    : {df['amount (INR)'].dtype}")
print(f"fraud_flag       : {df['fraud_flag'].dtype}")

```

```

DATA-TYPE VALIDATION
=====
Before cleaning:
timestamp      : object
amount (INR)   : int64
fraud_flag     : int64

After cleaning:
timestamp      : datetime64[ns]
amount (INR)   : int64
fraud_flag     : int64

```

```

# =====
# STAGE 4 – CONTAINER 3
# MISSING VALUES AND DUPLICATES
# =====

print("MISSING-VALUE VALIDATION")
print("=" * 60)

missing_counts = df.isnull().sum()

missing_table = pd.DataFrame({
    "Column": missing_counts.index,
    "Missing Values": missing_counts.values
})

display(missing_table)

```

```
total_missing = missing_counts.sum()

print(f"Total missing values: {total_missing:,}")

if total_missing == 0:
    print("✓ No missing values were found.")
else:
    print("⚠ Missing values require attention.")

# -----
# DUPLICATES
# -----

print("\nDUPLICATE VALIDATION")
print("=" * 60)

duplicate_count = df.duplicated().sum()

print(f"Duplicate rows: {duplicate_count:,}")

if duplicate_count > 0:
    df = df.drop_duplicates().copy()
    print(f"✓ Removed {duplicate_count:,} duplicate rows.")
else:
    print("✓ No duplicate rows required removal.")
```

MISSING-VALUE VALIDATION

=====

	Column	Missing Values	
0	transaction id	0	
1	timestamp	0	
2	transaction type	0	
3	merchant_category	0	
4	amount (INR)	0	
5	transaction_status	0	
6	sender_age_group	0	
7	receiver_age_group	0	
8	sender_state	0	
9	sender_bank	0	
10	receiver_bank	0	
11	device_type	0	
12	network_type	0	
13	fraud_flag	0	
14	hour_of_day	0	
15	day_of_week	0	
16	is_weekend	0	

Total missing values: 0

✓ No missing values were found.

DUPLICATE VALIDATION

=====

Duplicate rows: 0

✓ No duplicate rows required removal.

Next steps: [Generate code with missing_table](#) [New interactive sheet](#)

```
# =====
# STAGE 4 – CONTAINER 4
# NUMERICAL AND FRAUD VALIDATION
# =====

print("NUMERICAL AND FRAUD VALIDATION")
print("=" * 60)

negative_amounts = (
    df["amount (INR)"] < 0
).sum()

invalid_fraud_flags = (
    ~df["fraud_flag"].isin([0, 1])
).sum()

validation_table = pd.DataFrame({
    "Validation Check": [
        "Negative transaction amounts",
        "Invalid fraud flag values"
    ],
    "Count": [
        negative_amounts,
        invalid_fraud_flags
    ]
})
```

```

        "Count": [
            negative_amounts,
            invalid_fraud_flags
        ]
    })

display(validation_table)

if negative_amounts == 0:
    print("✓ No negative transaction amounts found.")
else:
    print("⚠ Negative transaction amounts found.")

if invalid_fraud_flags == 0:
    print("✓ All fraud flags contain valid binary values.")
else:
    print("⚠ Invalid fraud flag values found.")

```

NUMERICAL AND FRAUD VALIDATION

=====

	Validation Check	Count	
0	Negative transaction amounts	0	
1	Invalid fraud flag values	0	

✓ No negative transaction amounts found.

✓ All fraud flags contain valid binary values.

Next steps: [Generate code with validation_table](#) [New interactive sheet](#)

```

# =====
# STAGE 4 – CONTAINER 5
# CATEGORICAL VALUE VALIDATION
# =====

print("CATEGORICAL VALUE VALIDATION")
print("=" * 60)

transaction_types = (
    df["transaction type"]
    .dropna()
    .astype(str)
    .value_counts()
    .rename_axis("Transaction Type")
    .reset_index(name="Record Count")
)

transaction_statuses = (
    df["transaction_status"]
    .dropna()
    .astype(str)
    .value_counts()
    .rename_axis("Transaction Status")
    .reset_index(name="Record Count")
)

print("TRANSACTION TYPES")
display(transaction_types)

print("TRANSACTION STATUSES")
display(transaction_statuses)

```

CATEGORICAL VALUE VALIDATION

TRANSACTION TYPES

	Transaction Type	Record Count	
0	P2P	112445	
1	P2M	87660	
2	Bill Payment	37368	
3	Recharge	12527	

TRANSACTION STATUSES

	Transaction Status	Record Count	
0	SUCCESS	237624	
1	FAILED	12376	

Next
steps:

[Generate code with transaction_types](#)
[New interactive sheet](#)
[Generate code with transaction_statuses](#)

```
# =====
# STAGE 4 – FINAL VALIDATION
# =====

print("FINAL CLEANED DATASET")
print("=" * 60)

print(f"Final rows      : {df.shape[0]:,}")
print(f"Final columns   : {df.shape[1]}")

final_missing = df.isnull().sum().sum()
final_duplicates = df.duplicated().sum()
final_negative = (df["amount (INR)"] < 0).sum()
final_invalid_fraud = (~df["fraud_flag"].isin([0, 1])).sum()

final_validation = pd.DataFrame({
    "Validation Check": [
        "Missing values",
        "Duplicate rows",
        "Negative amounts",
        "Invalid fraud flags"
    ],
    "Final Count": [
        final_missing,
        final_duplicates,
        final_negative,
        final_invalid_fraud
    ]
})

display(final_validation)

print("\nSample of validated records:")
display(df.head())
```

FINAL CLEANED DATASET

Final rows : 250,000
Final columns : 17

Validation Check	Final Count	
0 Missing values	0	
1 Duplicate rows	0	
2 Negative amounts	0	
3 Invalid fraud flags	0	

Sample of validated records:

	transaction_id	timestamp	transaction_type	merchant_category	amount (INR)	transaction_status	sender_age_group
0	TXN0000000001	2024-10-08 15:17:28	P2P	Entertainment	868	SUCCESS	26
1	TXN0000000002	2024-04-11 06:56:00	P2M	Grocery	1011	SUCCESS	26
2	TXN0000000003	2024-04-02 13:27:18	P2P	Grocery	477	SUCCESS	26
3	TXN0000000004	2024-01-07 10:09:17	P2P	Fuel	2784	SUCCESS	26

Next steps:

[Generate code with final_validation](#)

[New interactive sheet](#)

Stage 4 Conclusion

The transaction dataset was validated through data-type, missing-value, duplicate, numerical, fraud-flag, and categorical checks.

The timestamp field was converted into a proper datetime format, while numerical fields required for validation were converted into appropriate numeric representations.

Missing values and duplicate records were explicitly checked, and duplicate records were removed only when present. Transaction amounts were checked for negative values, and fraud flags were validated against their expected binary values.

Transaction types and transaction statuses were also inspected to confirm the categorical values available for later grouping and analysis.

The final validation confirms the condition of the cleaned dataframe.

This validated dataframe is now ready to be passed to Stage 5 — Data Aggregation & Representation, where the cleaned transaction records will be summarized using grouping, counting, sorting, and aggregation operations.